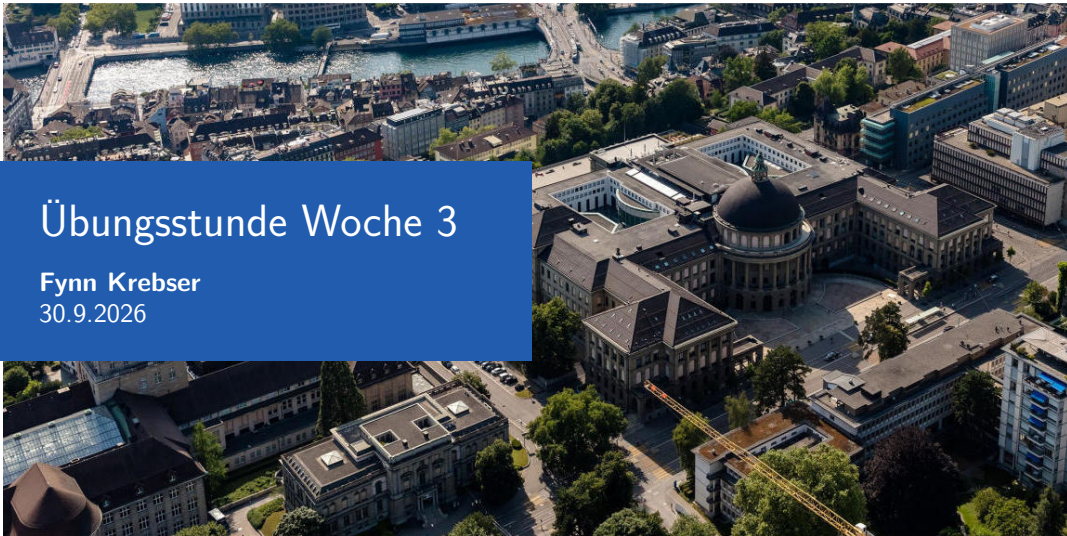


Übungsstunde Woche 3

Fynn Krebser

30.9.2026



Warmup Quiz

1. L- oder R-Value: $(x = 1) = 5$
2. Werte die folgenden Ausdrücke aus:
 - $5 < 4 < 1$
 - `true > false`
3. T/F: Lässt es sich leicht prüfen, ob eine unendliche Schleife in einem Programm existiert?
4. T/F: `1.0f + 2.0f == 3.0f`

Fragen letzter Woche: Pre- und Post Increment

- `x++` ist der Post-Increment Operator. Er erhöht den Wert von `x` um 1, gibt aber den alten Wert zurück.

Beispiel (bei initial `int x = 5;`):

– `int y = x++ + x;` ← *Hier ist x bereits 6! ($y = 5 + 6 = 11$)*

- `++x` ist der Pre-Increment Operator. Er erhöht den Wert von `x` um 1 und gibt den neuen Wert zurück.

Einschub: Mehr zu Modulo

- Erinnerung: $a \% b$ gibt den Rest der Division von a durch b zurück.
- Was passiert mit negativen Zahlen?
 - $-5 \% 2$ ergibt -1 in C++ ($(-2) \cdot 2 + (-1) = -5$).
 - $5 \% -3$ ergibt 1 in C++ (der Rest hat das gleiche Vorzeichen wie der Dividend).

Serie letzter Woche

Woche 1: Alles Tiptop!

Woche 2: Grösstenteils gut gelöst. Ein paar kleine Anmerkungen

- Ausdrücke sind als ganzes auch L- oder R-Values. (Aufgabe war evtl etwas unklar).
- Lest die Aufgaben immer genau. Teilweise sind die Fragen etwas versteckt.
- Denkt bei Vergleichsketten ($1 < 2 < 4 < 8$) an Typenkonvertierung.
- Ihr könnt abkürzen mit `int a,b,c` und `std::cin >> a >> b >> c;`.

Fragen?

Ziel von Heute

- For-Loops und If-Statements
- Debuggingstrategien
- Floats und Probleme damit verstehen

Scopes

- Definiert, wo Variablen gültig sind
- Variablen sind nur innerhalb des Scopes gültig.
- Merkregel: Immer wenn Klammern geschlossen werden, geht alles was darin definiert wurde verloren.

```
1 int main() {  
2     int x = 5;  
3     {  
4         int x = 3;  
5         std::cout << x << std::endl; // Ausgabe: ???  
6     }  
7     std::cout << x << std::endl; // Ausgabe: ???  
8     return 0;  
9 }
```

Scopes

```
for (int i = 0; i < 5; ++i) {  
    int last = i;  
}  
std::cout << last; // Does this compile?
```

For Schleifen

- Wird verwendet um Dinge mehrfach zu wiederholen

init **condition** **expression**



```
for ( int i = 0 ; i < 10 ; i++ ) {  
    std::cout << i << std::endl;  
}
```

1. Initialisierung
2. Bedingung überprüfen. Wenn Falsch, Schleife verlassen.
3. Schleifenrumpf ausführen
4. Ausdruck ausführen (z.B. `i++`)
5. Zurück zu Schritt 2

For Loop Tracing

- Was macht der folgende Code. Dokumentiere die Werte von `sum` und `i` in jeder Iteration.

```
1  int sum = 0;
2
3  for (int i = 0; i <= 3; ++i)
4      sum += i;
5
6  std::cout << sum << std::endl;
```

PythonTutor 

- Wie oft wird jede Schleife ausgeführt?
- `for (int i = 0 ; i < N ; i++)`
- `for (int i = 0 ; i <= N ; i++)`
- `for (int i = N ; i > 0 ; i /= 2)`

Largest Power

<https://expert.ethz.ch> 

If Statements

- Wird verwendet um Dinge nur unter bestimmten Bedingungen auszuführen
- Braucht ein boolsches Ergebnis, um zu entscheiden, ob der Code ausgeführt wird oder nicht.

```
if ( x > 0 ) {  condition  
    std::cout << "Positiv";  if-block (falls true)  
} else {  
    std::cout << "Nicht positiv";  else-block (falls false)  
}
```

If Statements

- Was macht der folgende Code. Dokumentiere die Werte von `n` in jeder Iteration mit Startwert `n = 13`.

```
1  int n;  
2  std::cin >> n;  
3  while (n != 1){  
4      if (n % 2 == 0) {  
5          n /= 2;  
6      } else {  
7          n = 3 * n + 1;  
8      }  
9  }
```

Debugging: Es regnet Sterne

Wie können wir sinnvoll Fehler finden in unserem Code?

```
1  const int n = 6;
2
3  // Compute  $n^{12}$ 
4  int prod = 1;
5  for (int i = 1; 1 <= i < 13; ++i) {
6      prod *= n;
7  }
8
9  // Output stars
10 for (int i = 1; i < prod; ++i) {
11     std::cout << "*";
12 }
13 std::cout << "\n";
```

Wir können den Code Schritt für Schritt durchgehen und die Werte von Variablen ausgeben, um zu sehen, wo etwas schief läuft.

Debuggingstrategien

- **Print Statements:** Füge Ausgaben in den Code ein, um die Werte von Variablen zu überprüfen, und zu wissen, wo etwas schief läuft.
- **Kleine Werte:** Teste den Code mit kleinen Eingabewerten, um das Verhalten zu verstehen. Häufig gibt es sogar Beispiele in der Aufgabenstellung.
- **CodeExpert:** Timeout beim Testen bedeutet idR, dass der Code in einer Endlosschleife hängt.

Strange Sum

<https://expert.ethz.ch> 

Alte Prüfungsfrage

Gegeben den folgenden Code: Sagen Sie, welche Aussagen wahr und welche falsch sind:
/ Given the following code snippet, say which statements are true and which are false:

```
1 #include <iostream>
2
3 int main() {
4     int a = 1;
5     int b = 0;
6     while (a < 3)
7         b += 10;
8         a += 1;
9
10    return 0;
11 }
```

True	False	
☐	☐	Nach der while Schleife, a == 2. / After the while loop, a == 2.
☐	☐	Nach der while Schleife, a == 3. / After the while loop, a == 3.
☐	☐	Nach der while Schleife, a == 4. / After the while loop, a == 4.
☐	☐	Nach der while Schleife, b == 20. / After the while loop, b == 20.
☐	☐	Nach der while Schleife, b == 30. / After the while loop, b == 30.
☐	☐	Die Schleife endet nie. / The loop never ends.

Floating Points Guidelines

1. Vermeidet das Testen von Gleichheit bei Floats.
2. Vermeide die Addition von Floats mit sehr unterschiedlichen Größenordnungen.
3. Vermeide die Subtraktion von Floats, die sehr nahe beieinander liegen.

Nun soll es um Intuition gehen, die Details kommen Woche 8TM.

Floats und Gleichheit

- Problem ähnlich zu wieso $\frac{1}{3} * 3 = 0.99999 \neq 1$

```
1 float a = 2.0f;
2 if (std::sqrt(a) * std::sqrt(a) == a) {
3     std::cout << "Gleich!" << std::endl;
4 } else {
5     std::cout << "Nicht gleich!" << std::endl;
6 }
```

Floats und Addition

- Was ist $1 + 0.0000001$?

$$1 = 1.0$$

Mathematicians

Physicists



Wie vergleicht man Floats?

- Erste Idee: Kleine Differenz ϵ definieren und testen, ob $|a - b| < \epsilon$.

```
1 float epsilon = 0.0001f;  
2 if (std::abs(a - b) < epsilon) {  
3     std::cout << "Gleich!" << std::endl;  
4 }
```

- Beispiel: $a = 10.0$, $b = 12 \Rightarrow |a - b| = 2 > \epsilon \Rightarrow$ nicht gleich.

Probleme?

- $a = 10.0$, $b = 10.00001 \Rightarrow |a - b| = 0.00001 < \epsilon \Rightarrow$ gleich, obwohl sie sich unterscheiden.
- Bessere Lösung: Relative Differenz testen, d.h. $|a - b| < \epsilon \cdot \max(|a|, |b|)$

Tipps zu den Übungen

Fibonacci Primes

- Verwende den Primzahltest aus der Vorlesung.
- Verwende eine Schleife, um die Fibonacci-Zahlen zu generieren.
- Teste jede Fibonacci-Zahl auf Primzahl.

Fibonacci Overflow

- Wann entsteht ein Overflow? - Stelle die Formel um.